01 · ACCESS

Getting started on the Illinois Campus Cluster

A practical workflow for data analysis



Illinois Computes · Official photograph

One small analysis from input to result

Input: data/samples.csv

```
sample_id,reads
sample_A,1000
sample_B,1500
sample_C,1200
```

Expected result: results/summary.csv

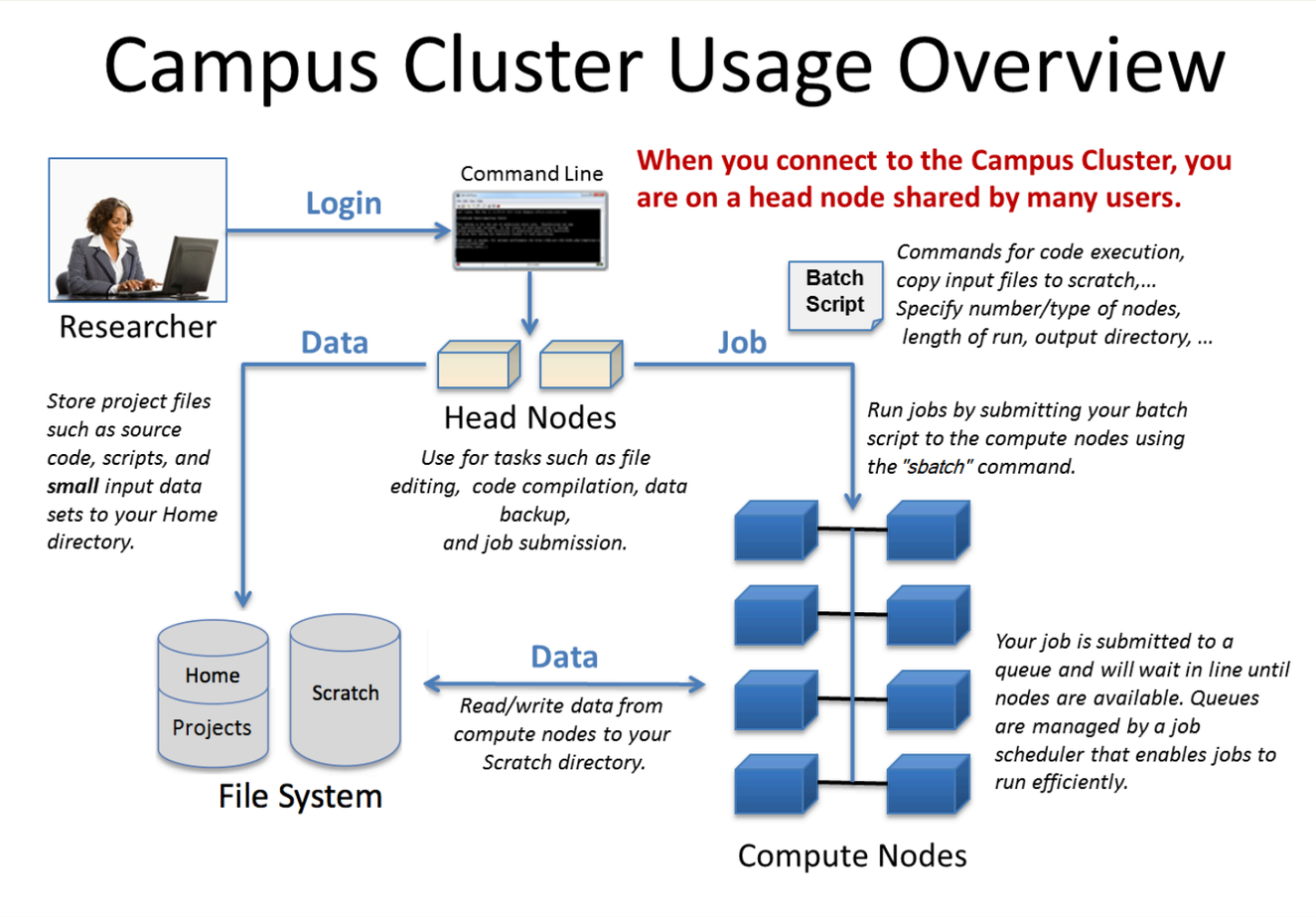
```
samples,total_reads
3,3700
```

The program analysis.py reads the table and adds the reads.

The job file analysis.sbatch asks Slurm, the job scheduler, to run it.

> We will move this project onto the cluster and produce this result.

Your laptop, the login node, and the compute node



HPC = high-performance computing. A node is one computer. Login: prepare files. Compute: run scheduled analysis.

Account access and the details you need

Existing lab access

Ask the PI or technical contact which investor group to join.

A new computing need

Start an Illinois Computes request.

Keep this detail	Used for
NetID	Your login identity
Slurm account	The group’s compute allocation
Partition	A queue you may submit to
Project path	An approved writable folder

Research access request ↗ Illinois Computes request ↗

Wait for approval and confirm these details and Duo access before connecting.

The first SSH login

On your laptop: open Terminal (macOS) or PowerShell (Windows)

```
ssh YOUR_NETID@cc-login.campuscluster.illinois.edu
```

`ssh` starts the connection · before `@`: your user name · after `@`: the cluster address

1 · Run the command

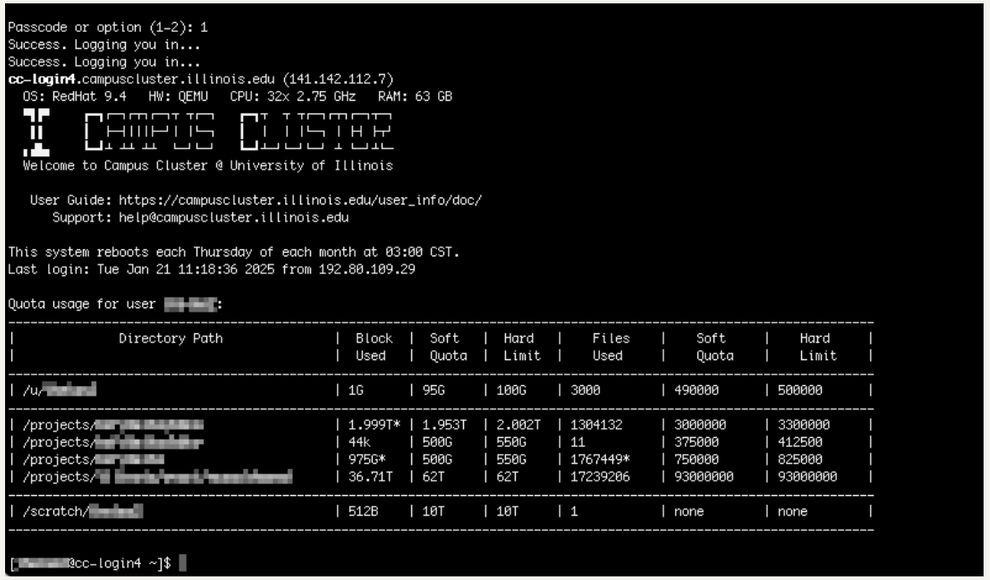
Replace YOUR_NETID, then press Enter.

2 · Password

Type your NetID password and press Enter; typing stays hidden.

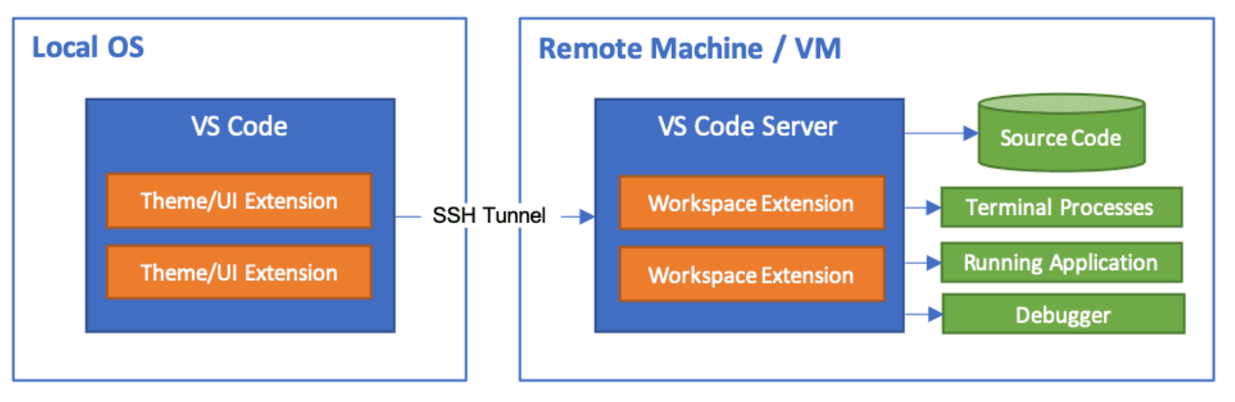
3 · Duo

Follow the prompt and approve the authentication request.



A welcome message and remote prompt indicate that login succeeded. Banner values may be historical.

The local editor and the remote project



Original Microsoft figure. Focus on the local editor, SSH connection, and remote project.

On your laptop The VS Code window and its controls

On the cluster The project files and remote terminal processes

> Remote editing gives you a familiar interface to the files on the cluster.

Local and remote: where things happen

LOCAL

Your laptop

VS Code window

View and edit

`analysis.py`

Your screen, keyboard, and mouse

Remote - SSH



View · edit · save

REMOTE

The cluster

Project files

`demo/analysis.py`

Save updates this file.

Remote terminal

Commands run on the cluster.

In the SSH window, you work with files and a terminal on the cluster.

Prepare VS Code on your laptop

Install VS Code	Download the desktop application for your operating system.
Open Extensions	Click the Extensions icon in VS Code’s left sidebar.
Search	Enter Remote - SSH; choose the extension published by Microsoft.
Install	Click Install and wait until it finishes.

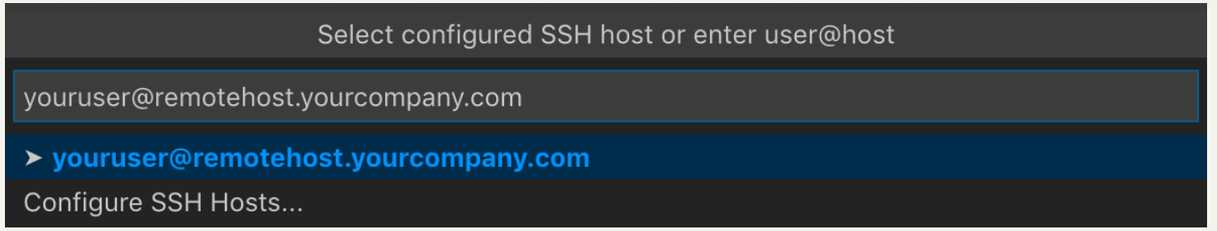
Official VS Code setup ↗

> This installs the editor and connection tool on your laptop.

Microsoft · VS Code setup · Microsoft · Remote - SSH

Connecting VS Code with Remote - SSH

Command Palette	Cmd+Shift+P on macOS; Ctrl+Shift+P on Windows/Linux
Choose	Remote-SSH: Connect to Host...
Enter and sign in	YOUR_NETID@cc-login.campuscluster.illinois.edu



Microsoft example UI. Enter your NetID and the ICC hostname in this field.

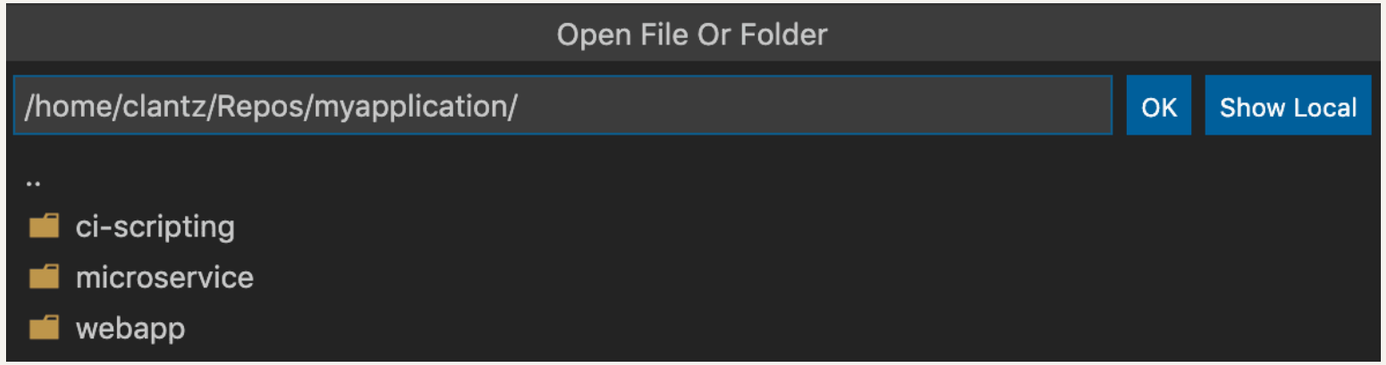
Complete password + Duo. Choose Linux if asked. The lower-left indicator should show SSH and the host.

Choose where the project will live

Location	Use it for	Remember
/u/YOUR_NETID	Settings and small personal files	Home has a quota.
/projects/YOUR_GROUP	Group data, software, and retained outputs	Use your assigned writable location.
/scratch/YOUR_NETID	Temporary working data	Files can be purged.

> For this example: `/projects/YOUR_GROUP/demo`

Open the remote project location



Microsoft example UI. Replace its sample path with your assigned project location.

File → Open Folder	Open /projects/YOUR_GROUP or your assigned path.
Check the connection	The SSH indicator should identify the cluster.
Find the destination	This is where the demo folder will be uploaded.

> The Explorer now shows the remote location, before the upload.

Upload the demo folder with VS Code

On the laptop	Unzip the offline package; find HPC_Introduction/demo.
Upload	Drag demo into the remote Explorer destination.
Open the project	File → Open Folder → /projects/YOUR_GROUP/demo
Check a file	Open data/samples.csv and look for the three sample rows.

> Wait for the transfer to finish. The input and scripts now live on the cluster.

The files now in the remote demo folder

Uploaded files

```
demo/
  data/samples.csv
  analysis.py
  analysis.sbatch
  hello.sbatch
  notes.txt
  README.md
```

data/samples.csv

The same three-row input table

analysis.py

The calculation we want to run

hello.sbatch

A first job that checks submission

analysis.sbatch

The job that will run the calculation

> The results folder and Python environment will be created in later steps.

Open a terminal in the same project

VS Code → Terminal → New Terminal

```
hostname
cd /projects/YOUR_GROUP/demo
pwd
```

hostname	Check which computer is running the shell.
cd	Use the same remote demo folder as Explorer.
pwd	Confirm the current folder before running commands.

> Run one line, press Enter, wait for the prompt, then continue.

Reading a command instead of memorizing it

```
head -n 5 data/samples.csv
```

Program	head displays the beginning of a text file.
Option + value	-n 5 asks for the first five lines.
File path	data/samples.csv names the input.

> Quotes keep a path together: `head -n 5 "data/samples.csv"`. They are required for paths containing spaces.

Navigate the uploaded project

Transcript: type commands after $; other lines are output

```
$ pwd
/projects/YOUR_GROUP/demo
$ cd data
$ pwd
/projects/YOUR_GROUP/demo/data
$ cd ..
$ pwd
/projects/YOUR_GROUP/demo
$ cd ~
$ pwd
/u/YOUR_NETID
$ cd -
/projects/YOUR_GROUP/demo
$ pwd
/projects/YOUR_GROUP/demo
```

Command	Meaning
pwd	Print the current folder
cd data	Enter the data subfolder
cd ..	Go up one folder
cd ~	Go to your home folder
cd -	Return to the previous folder

`cd -` also prints its destination. The next `pwd` confirms it.

`.` current folder · `..` parent · `/projects/…` absolute path · `data/…` relative path

List files and inspect their details

Command	What changes
ls	List visible names
ls -a	Include dot-prefixed entries
ls -l	Show details such as size and date
ls -lh	Use readable sizes, such as K or M
ls -lah	Combine all three options

Illustrative listing lines; total lines omitted

```
$ ls data
samples.csv
$ ls -a data
.  ..  samples.csv
$ ls -l data
-rw-r--r-- 1 user group 58 Oct 27 10:00 samples.csv
$ ls -lh data
-rw-r--r-- 1 user group 58 Oct 27 10:00 samples.csv
$ ls -lah data
drwxr-xr-x 2 user group 4.0K Oct 27 10:00 .
drwxr-xr-x 3 user group 4.0K Oct 27 10:00 ..
-rw-r--r-- 1 user group   58 Oct 27 10:00 samples.csv
```

The CSV is only **58 bytes**, so `-h` still shows 58. Later, `ls -a` will reveal the `.venv` folder we create.

Creating, copying, moving, and deleting

Command	Result
`mkdir results`	Create one new folder
`mkdir -p results/qc`	Create missing parent folders; allow existing folders
`cp -i notes.txt notes_backup.txt`	Copy a file; ask before overwriting
`cp -r data data_backup`	Copy a folder and its contents to a new folder
`mv -i notes_backup.txt notes_old.txt`	Rename or move; ask before overwriting
`rm -i notes_old.txt`	Ask before deleting this file

`cp` and `mv`: source first, destination second. Shell deletion normally has no Trash recovery.

Inspecting a small text file

Command	Use
`cat notes.txt`	Print the entire small file
`head -n 5 data/samples.csv`	Read the header and first rows
`less notes.txt`	Browse text; press q to leave

Output of head for the included sample table

```
sample_id,reads
sample_A,1000
sample_B,1500
sample_C,1200
```

> Preview a few rows before running the analysis. Avoid printing a huge file with `cat`.

Find a file, then find text inside it

Search filenames within data and its subfolders

```
find data -type f -name "*.csv"
```

-type f	Match files; -name tests their names against the quoted pattern.
"*.csv"	The * matches any text before .csv.

Search inside this file; -n includes line numbers

```
grep -n "sample_B" data/samples.csv
```

Expected grep output: line 3 contains the match

```
3:sample_B,1500
```

Count lines and check disk usage

Run this command

```
wc -l data/samples.csv
```

Expected output; do not type this line

```
4 data/samples.csv
```

wc -l

Count lines in this text file.

4 lines

One header plus three sample rows.

Check space used by the input folder

```
du -sh data
```

`-s` gives one summary total · `-h` uses readable units such as K or M

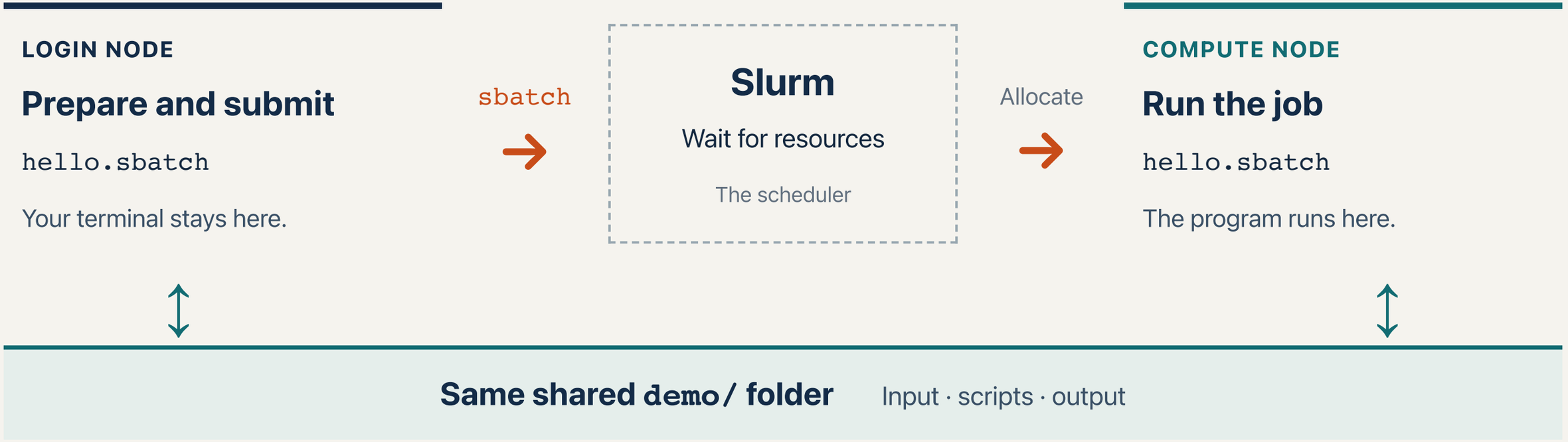
> Line count is not automatically sample count. Disk usage is not your remaining quota.

Help and everyday terminal shortcuts

Command / key	Use when needed
`man ls`	Open the manual; press q to leave
`head --help`	Show the Linux command's built-in help
Tab	Complete a file or folder name
↑ / `history`	Recall previous commands / list shell history
Ctrl+C	Interrupt the current foreground command
`clear` / `exit`	Clear the display / leave the current shell

Keep this shell open for the next step. Use exit only when you intend to leave it.

Slurm runs work on allocated resources



Batch: submit a script. **Interactive:** request a compute shell with `srun`.

> Submit on the login node; the job runs on an allocated compute node.

A small batch script: hello.sbatch

```
01 #!/bin/bash
02 #SBATCH --job-name=hello
03 #SBATCH --account=YOUR_ACCOUNT
04 #SBATCH --partition=YOUR_PARTITION
05 #SBATCH --nodes=1
06 #SBATCH --ntasks=1
07 #SBATCH --cpus-per-task=1
08 #SBATCH --mem=1G
09 #SBATCH --time=00:05:00
10 echo "My first HPC job"
```

01 Use Bash to read the commands.

02 Give the job a recognizable name.

03 Choose your assigned allocation.

04 Choose an eligible queue.

05 Request one compute node.

06 Request one task for this program.

07 Reserve one CPU for that task.

08 Request 1 GiB of RAM per node.

09 Set a five-minute running-time limit.

10 Print a message into the job output.

Edit these lines inside hello.sbatch in VS Code, then save. Submit it from the terminal after reviewing the resource settings.

Nodes, tasks, CPUs, memory, and time

Example resource request: count the node, the tasks, then the CPUs.

`--nodes=1`

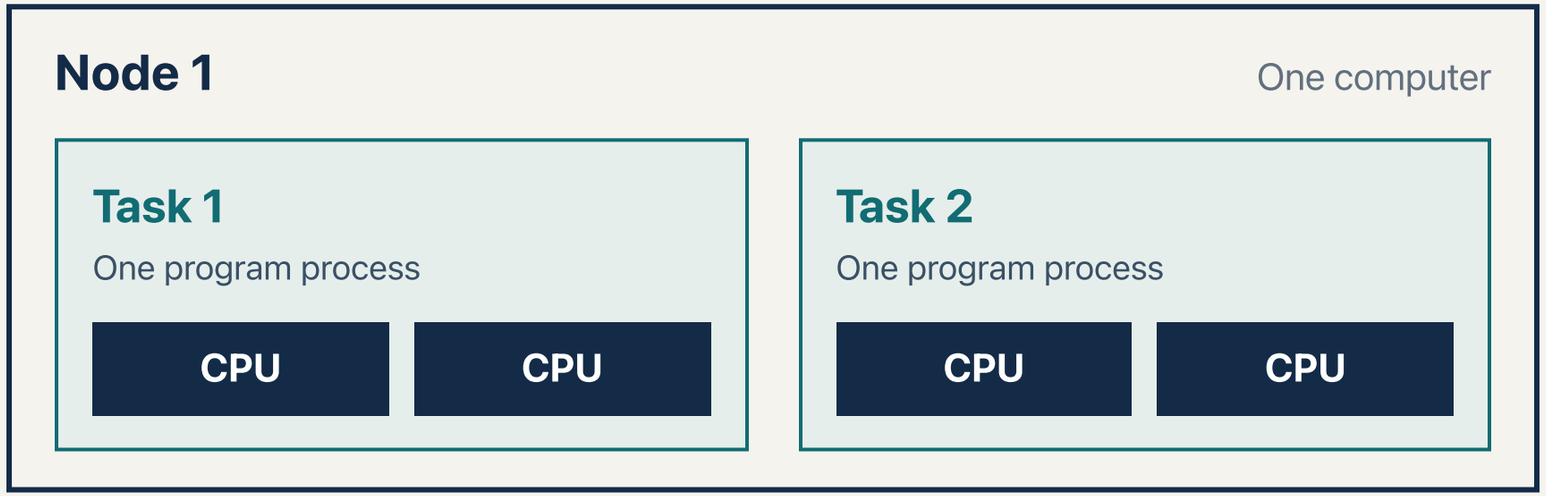
1 computer

`--ntasks=2`

2 tasks in the whole job

`--cpus-per-task=2`

2 CPUs reserved for each task



> **2 tasks × 2 CPUs/task = 4 CPUs requested**
>
> ntasks is already the total; do not multiply by nodes again.

Reserving resources does not launch the tasks or make the program parallel.

Keep hello.sbatch at: 1 node · 1 task · 1 CPU/task

`--mem=1G` 1 GiB of RAM per node

`--time=00:05:00` Runtime limit: hours:minutes:seconds

Submit the first job: hello.sbatch

In the remote demo folder, after saving the file

```
pwd
ls hello.sbatch
sbatch hello.sbatch
```

Illustrative response: record this job ID

```
Submitted batch job 123456
```

> This first job prints a message. It does not run analysis.py yet.

Checking the queue and stopping a job

-u selects your user; run squeue again to refresh the snapshot

```
squeue -u YOUR_NETID
```

State	Meaning
PD	Pending: waiting for resources or another scheduling condition
R	Running: the allocation has started

Only for an unwanted job; let hello finish in this example

```
scancel 123456
```

When hello leaves the queue, continue to its output and accounting record.

Check the first job's message

-j selects the job; --format chooses the displayed columns

```
sacct -j 123456 --format=JobID,State,ExitCode
```

Illustrative successful record

```
JobID    State        ExitCode
123456   COMPLETED    0:0
```

`0:0` means exit status 0 and no terminating signal. Use your actual job ID.

Expected text: My first HPC job

```
cat slurm-123456.out
```

> Continue when the job has completed and the expected message is present.

When the analysis needs a GPU

Optional GPU job: use a valid partition and configured GPU type

```
#SBATCH --partition=YOUR_GPU_PARTITION
#SBATCH --gres=gpu:GPU_TYPE:1
```

Inside an allocated NVIDIA GPU job

```
nvidia-smi
```

--gres	Request a generic resource; here gpu:TYPE:COUNT.
GPU_TYPE / final 1	The configured GPU type / one GPU per allocated node.

> The tiny demo needs no GPU. Add this only for software that supports and uses one.

When a particular node is required

Optional: use an actual eligible node name

```
#SBATCH --nodes=1
#SBATCH --nodelist=NODE_NAME
```

sinfo: inspect resources; -p selects the partition; -N lists nodes

```
sinfo -p YOUR_PARTITION -N
```

--nodes=1	How many nodes you request
--nodelist	Which named node you request

> Keep this out of the demo job. Name a node only for a specific need agreed with your group.

Get a compute shell for software setup

From the login shell in demo: paste this whole block as one command

```
srun --account=YOUR_ACCOUNT --partition=YOUR_PARTITION \
  --nodes=1 --ntasks=1 --cpus-per-task=2 \
  --mem=4G --time=00:15:00 --pty bash
```

srun	Start a program on allocated compute resources.
--pty bash	Open an interactive terminal running the Bash shell.
After it starts	Run hostname and pwd. Keep this shell open for Python setup.

A trailing `\` continues a command onto the next line; nothing follows it on that line.

Make Python available in the compute shell

Inside the allocation, in demo; use a version shown as available

```
module avail python
module load python/3.11.11
module list
```

Command	Meaning
module avail python	List cluster modules matching Python
module load python/3.11.11	Make this Python version available in this shell
module list	Confirm which cluster modules are loaded

> analysis.py imports pandas. Next, create a place to install that package.

Create .venv and install pandas

Continue in the same compute shell and remote demo folder

```
python -m venv .venv
source .venv/bin/activate
python -m pip install pandas
ls -a
```

Create .venv	-m runs Python’s venv utility; .venv is the new folder.
Activate	source applies the activation script to this current shell.
Install pandas	-m pip runs pip with this Python; install names the action.

Wait for pip to finish and the prompt to return. Then check the import on the next slide.

Verify pandas and record its version

-c executes the quoted Python code immediately

```
python -c "import pandas; print(pandas.__version__)"
```

import pandas	Check that this Python can load the package.
Print version	print(pandas.__version__) displays the installed version; expect a version number.

freeze lists installed package versions; > writes them into this file

```
python -m pip freeze > requirements.txt
```

> Continue only after the import succeeds. Keep this shell open for the next slide.

The environment stays; activation is per shell

Install packages

```
python -m pip install pandas
```

After creating and activating .venv

Write files →

Shared project storage

`demo/.venv/`

Python + installed packages

These files remain.

Activate in each new shell or job

```
source .venv/bin/activate
```

Current compute shell

`python` → `.venv/bin/python`

Same existing **.venv** No reinstall

A later batch job

`python` → `.venv/bin/python`

> Closing a shell keeps .venv. A new shell or job selects the environment again.

Return to the login shell

Exit the allocated shell once, then check where you are

```
exit
hostname
pwd
```

hostname	Should identify the original login host again.
pwd	Should still identify /projects/YOUR_GROUP/demo.
If the folder differs	cd back to your approved demo path.

> The Python project is ready for submission. R and Conda are alternative routes for other projects.

For an R project: prepare a package library

Alternative route: use a separate allocated compute shell

```
module load R
export R_LIBS_USER="$PWD/R-library"
mkdir -p "$R_LIBS_USER"
R
```

Syntax	Meaning
export NAME=value	Make a setting available to programs launched by this shell
$PWD / $R_LIBS_USER	Substitute the current folder / the library path just set
R	Start the R console; its prompt changes to >

> The next slide’s commands go at the R prompt, not the shell prompt.

For an R project: install the packages

Inside R: a CRAN package

```
install.packages("ggplot2",
  repos = "https://cloud.r-project.org")
library(ggplot2)
```

Inside R: a Bioconductor package

```
install.packages("BiocManager",
  repos = "https://cloud.r-project.org")
BiocManager::install("DESeq2",
  ask = FALSE, update = FALSE)
```

`repos` : download repository · `::` : use a package's function

`ask = FALSE` : skip update questions · `update = FALSE` : leave unrelated packages unchanged

After installation: leave R without saving an R workspace

```
q(save = "no")
```

Alternative: use a supplied Conda environment

Alternative route when the project supplies environment.yml

```
module avail miniconda3
module load YOUR_CONDA_MODULE
conda env create -f environment.yml
conda run -n analysis python -c "import pandas; print(pandas.__version__)"
```

environment.yml	Records the environment name and requested packages
-f environment.yml	Read the environment specification from this file.
-n analysis	Use the environment named analysis (from name: analysis in the file).

Use a separate allocated setup session. Expect a pandas version; this replaces the .venv route.

Prepare analysis.sbatch for the same project

1 · Beginning of analysis.sbatch

```
#!/bin/bash
#SBATCH --job-name=analysis
#SBATCH --account=YOUR_ACCOUNT
#SBATCH --partition=YOUR_PARTITION
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
```

2 · Continue in the same file, after the left column

```
#SBATCH --output=analysis-%j.out
#SBATCH --error=analysis-%j.err

set -e
module load python/3.11.11
source .venv/bin/activate
python analysis.py
```

> set -e stops this script if setup fails. Replace placeholders and save the file before submitting.

Submit the analysis job

In the login shell, from the demo folder

```
pwd
ls data/samples.csv analysis.py analysis.sbatch
sbatch analysis.sbatch
```

Illustrative analysis job ID; hello used 123456

```
Submitted batch job 123457
```

Check whether this new job is pending or running

```
squeue -u YOUR_NETID
```

> Use this new job ID for the analysis logs and completion checks.

Follow the analysis log

After the analysis job starts and the log exists

```
tail -n 20 analysis-123457.out
tail -f analysis-123457.out
```

First press Ctrl+C to stop tail; then search the error log

```
grep -in "error" analysis-123457.err
```

Ctrl+C	Stop tail; the submitted job keeps running.
Need to cancel?	scancel 123457 cancels this analysis job.

> tail: -n 20 = last 20 lines, -f = follow new output · grep: -i = ignore case, -n = line numbers

Check the state and the result file

After the analysis job finishes

```
sacct -j 123457 --format=JobID,State,ExitCode
cat results/summary.csv
```

Expected file contents

```
samples,total_reads
3,3700
```

Job record

COMPLETED and ExitCode 0:0

Output log

Saved results/summary.csv

Input comparison

3 rows; 1000 + 1500 + 1200 = 3700

> The result should match the goal shown at the start of the lecture.

Open the result in VS Code

Explorer → results	Open summary.csv and check 3 samples and 3700 total_reads.
Keep it remote	Read or edit the file without downloading it first.
Need a local copy?	Right-click summary.csv and choose Download...

Other output types	Viewer
Logs / PNG / JPEG	Built-in text or image viewer
CSV / TSV	Text editor; Rainbow CSV adds column colors
PDF / notebooks	A PDF viewer extension / Microsoft Jupyter viewer

Microsoft · Remote - SSH · Rainbow CSV · Extension documentation · vscode-pdf · Extension documentation · Microsoft · Jupyter extension · Microsoft · File import and download implementation

For an R project: change the job body

R commands below the #SBATCH resource settings

```
module load R
export R_LIBS_USER="$PWD/R-library"
Rscript analysis.R
```

After copying and configuring the R templates from examples/

```
sbatch analysis_R.sbatch
```

Same input	data/samples.csv
R output	results/summary_R.csv: 3 samples and 3700 total_reads

> Use the same R version and library location used during package setup.

The same HPC routine for different analyses

Project	What changes	What stays the same
Bulk analysis	Inputs, analysis script, packages	Storage → environment → job → outputs
Single-cell analysis	Matrices, metadata, tool-specific settings	Storage → environment → job → outputs
Spatial analysis	Matrices, images, coordinates, tool-specific settings	Storage → environment → job → outputs

> Start with the project's working script and a small test input. Scale after checking the result.

Common errors and the first useful check

What you see	Check first
No such file or directory	pwd, the path, and the filename’s capitalization
Command not found	The required module or environment
No module named… / no package called…	The active Python or R library, then installation
OUT_OF_MEMORY / TIMEOUT	Requested RAM or runtime and the tool’s actual needs
Invalid account / partition	The allocation and queue pair assigned to you
Permission denied / quota exceeded	The target folder’s access and available quota

The completed project

Key files after the successful example run

```
demo/
  data/samples.csv
  analysis.py
  analysis.sbatch
  .venv/
  requirements.txt
  analysis-123457.out
  analysis-123457.err
  results/summary.csv
```

Started with

A small input table and saved scripts

Prepared

The remote folder and pandas environment

Submitted

analysis.sbatch through Slurm

Verified

3 samples and 3700 total reads

> Keep the inputs, script, environment record, job file, and checked outputs together.

Documentation and help

ICC · Accessing the System

ICC · Storage and Data

Missing Semester · Introduction to the Shell (2026)

Microsoft · Remote - SSH

Python Packaging · pip and venv

Conda · Managing environments

ICC · Running Jobs

ICC · Software and Modules

Missing Semester · Command-line Environment (2026)

SchedMD · sbatch

Bioconductor · Installation

Reveal.js · Vertical slides

ICC support: **help@campuscluster.illinois.edu**
Full references and image credits are in SOURCES.md.